

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for the Dehallucination Add-On Module for Enhancing Reliability of AI Applications. The module is intended to reduce incorrect or unsupported statements produced by AI applications by checking factual claims against trusted sources or knowledge graphs before returning a final response.

The SRS covers the proposed software module, its users, interfaces, major processing steps, functional requirements, nonfunctional requirements, assumptions, and traceability. It is written for the Phase-1 mini-project and describes the expected behavior of the system before implementation.

1.2 Intended Audience and Reading Suggestions

This document is intended for the student project team, project guide, developers, testers, reviewers, and other readers who need to understand how the proposed system should work.

- Read Section 1 first for the purpose and scope of the project.
- Section 2 gives the overall architecture, functions, users, environment, and constraints.
- Section 3 describes the interfaces used by the module.
- Section 4 contains the analysis models.
- Section 5 contains the detailed functional requirements.
- Section 6 contains nonfunctional requirements such as performance, security, reliability, and usability.
- The appendices provide terminology, field information, and requirement traceability.

1.3 Product Scope

The proposed product is a software add-on that can be placed between an AI application and its final response delivery path. It accepts AI-generated text, identifies factual claims, searches trusted information sources, compares the claims with the retrieved evidence, and produces a verification result.

The module is not intended to replace the underlying AI model. Its purpose is to provide an additional verification layer that can identify potentially hallucinated content, show supporting evidence, assign confidence information, and return a verified or modified response.

- Accept generated text from a chatbot or content generator.
- Identify factual claims in the generated response.
- Retrieve relevant information from configured trusted sources.
- Compare claims with retrieved evidence.
- Classify claims as supported, contradicted, or uncertain where possible.
- Generate a verification result and confidence information.
- Return a verified or modified response with supporting evidence.
- Maintain logs and verification records for review.

1.4 References

- PES University, Department of Computer Science and Engineering, JACKFRUIT - SE MINI-PROJECT GUIDELINES 2026, Software Requirements Specification template.
- Problem Statement 39: Dehallucination Add-On Module for Enhancing Reliability of AI Applications.

Note: Detailed external implementation references may be added during later phases when the exact AI model, retrieval framework, knowledge graph, database, and APIs are finalized.

2. Overall Description

2.1 Product Perspective

The proposed system is a new, self-contained verification add-on that can be integrated with an existing AI application such as a chatbot or content generator. The AI application sends its generated response to the dehallucination module. The module processes the response, retrieves evidence, verifies factual claims, and sends the result back to the application.

The main internal flow is: input processing → claim extraction → evidence retrieval → fact verification → result generation. Trusted external sources provide evidence, while a database can store requests, results, source information, and logs.

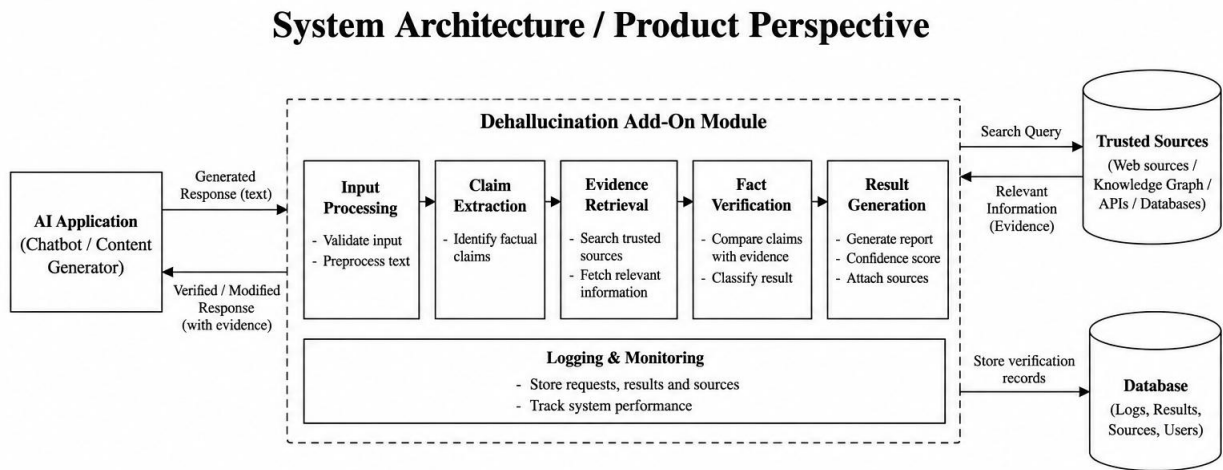


Figure 1: System Architecture of the Dehallucination Module

Figure 1: System Architecture / Product Perspective

2.2 Product Functions

- Receive and validate AI-generated responses.
- Preprocess text before verification.
- Extract factual claims from the response.
- Retrieve relevant evidence from trusted sources or knowledge graphs.
- Compare claims with evidence using a verification engine.
- Calculate or assign a confidence score to verification results.
- Flag unsupported or contradicted claims.
- Generate a verified/modified response and verification report.
- Display supporting evidence and sources.
- Provide an API for integration with other AI applications.
- Manage configured trusted sources.
- Store logs and verification records for review.

2.3 User Classes and Characteristics

- End User – uses the connected chatbot/content generator and views the verified response and evidence. Basic technical knowledge is expected.
- Developer/Integrator – integrates the verification module or API with an AI application. Familiarity with APIs and software integration is expected.
- Administrator – manages trusted sources, configuration, and system logs. Requires higher privileges than a normal user.
- Reviewer – examines flagged or uncertain results and supporting evidence for evaluation and improvement.

2.4 Operating Environment

The system is expected to operate as a software service on a computer or server capable of running the selected programming language, AI/NLP libraries, retrieval components, and database. A web-based interface or API may be used for integration with the target AI application.

- Operating system: Windows or Linux environment suitable for development/deployment.
- Application environment: Python-based or equivalent backend environment.
- Database: relational or document database depending on implementation choice.
- Network: Internet or local network connection when external trusted sources are used.
- Browser: modern web browser for any web-based user interface.
- External services: trusted web sources, APIs, or knowledge graphs as configured.

2.5 Design and Implementation Constraints

- The module must work as an add-on and should not require replacement of the underlying AI model.
- Verification quality depends on the availability and quality of trusted evidence.
- External source rate limits, API availability, and network connectivity may affect retrieval.
- The implementation must clearly separate generated content from retrieved evidence.

- The system should avoid treating unsupported generated text as verified merely because no contradiction was found.
- Authentication and access control are required for administrative functions.
- The exact AI model, database, and source APIs may be finalized during implementation.

2.6 Assumptions and Dependencies

- The connected AI application can provide its generated text to the module.
- At least one trusted source or knowledge source is available for verification.
- The system has network access when external sources are required.
- External APIs or knowledge graphs used by the implementation remain accessible.
- The project team will define source reliability rules during implementation.
- The final confidence-scoring method may depend on the selected verification approach.

3. External Interface Requirements

3.1 User Interfaces

The user interface should be simple and focused on verification. A typical verification screen should allow the user to submit or receive an AI-generated response and then view the verification result.

- Text input or API-based submission of an AI-generated response.
- Verify button or equivalent action.
- Verification status for the response and individual claims.
- Confidence information.
- Supporting evidence and source references.
- Indication of unsupported, contradicted, or uncertain claims.
- Error messages when input or source retrieval fails.
- Administrator screens for source management and logs, if implemented.

3.2 Software Interfaces

The module interfaces with the AI application, claim extraction/NLP components, retrieval components, verification engine, database, and trusted external sources.

- AI application interface: receives generated text and returns a verification result.
- Claim extraction component: receives text and returns factual claims.
- Evidence retrieval component: receives claims/search queries and returns relevant evidence.
- Verification engine: receives claims and evidence and returns verification status and confidence information.
- Database interface: stores and retrieves verification records, source details, and logs.
- Trusted-source interfaces: access web sources, APIs, databases, or knowledge graphs.

3.3 Communications Interfaces

Communication between the AI application and the module may use HTTP/HTTPS APIs. JSON is the expected message format for structured requests and responses. HTTPS should be preferred whenever data is transferred over an external network.

- Request: response text, request identifier, and optional configuration information.
- Response: verification status, modified/verified response, claim results, confidence information, and evidence/source information.
- External retrieval: HTTP/HTTPS or the protocol required by the selected trusted source.
- Communication failures should be reported without presenting incomplete verification as successful verification.